

Keywords

Refundable, Solidity, Token, EVM
GPG: AA80AF873B20976E

Email Correspondence

hugo.card@usp.br

<https://github.com/BelleMoon/RFT>Hugo Cardoso Ferreira de Araújo¹

Refundable Token - A no intermediaries time-ensurance eletronic refund system

Abstract

This article introduces a Refundable Token (RFT) framework to address temporal uncertainty in blockchain transaction finality. By assigning a dynamic minimal refund block number to each address, our system enables a fixed time-window for transaction reversibility, defined by the blockchain's current block number. A built-in debt-check mechanism secures refundable amounts and prevents premature transfers (i.e., "Send Before Refund" attacks). Refund data is managed via a centralized storage mapping, ensuring constant-time ($\mathcal{O}(1)$) operations. This decentralized, intermediary-free smart contract paradigm provides a robust, time-ensured electronic refund system, enhancing both security and operational predictability in cryptocurrency transactions – creating an environment to a no intermediaries time-ensured electronic refund system.

Introduction

Exigence and Rationale

Blockchain technology has revolutionized digital transactions by enabling secure, decentralized, and immutable record-keeping. Cryptocurrencies, as one of the primary applications of blockchain, facilitate peer-to-peer transactions without the need for intermediaries. However, one of the critical limitations of most cryptocurrency networks is the irreversibility of transactions. Once a transaction is confirmed and added to the blockchain, it becomes final, making it difficult to rectify accidental transfers, fraudulent activities, or disputed payments.

This paper introduces the Refundable Token (RFT), a novel cryptocurrency token standard designed to enable time-constrained, issuer-approved refunds. The RFT protocol integrates a structured refund mechanism that allows users to reclaim sent funds within a predefined time window. This mechanism is implemented through smart contracts that record refundable transactions, track refund deadlines, and ensure that refunds can only be processed by the original sender within the allowed time frame. The design of RFT aims to enhance user security, reduce transaction risks, and improve trust in decentralized financial ecosystems.

The key innovation behind RFT lies in its ability to facilitate refundable transactions without compromising the fundamental principles of decentralization and security. By introducing a minimal refund block constraint, the protocol ensures that all refund requests adhere to a time-based validation system, thereby preventing unauthorized or indefinite claims. Additionally, the smart contract architecture supports transparent debt management, allowing recipients to track and settle outstanding obligations dynamically.

This paper explores the architecture, implementation, and security implications of the RFT protocol. We analyze its potential applications in e-commerce, subscription-based services, and decentralized financial systems, where transaction reversibility can enhance consumer protection and financial flexibility. Furthermore, we evaluate the impact of RFT

on network efficiency, smart contract security, and user adoption within blockchain ecosystems.

Moreover, the RFT protocol is engineered with a strong emphasis on computational efficiency. Key operations such as transaction initiation, refund processing, and debt clearance are executed in constant time $\mathcal{O}(1)$, independent of the system's scale. This is accomplished by leveraging Solidity's direct mapping and fixed-size array structures, which ensure that each operation requires a predictable and minimal amount of computational resources.

Justification

In the realm of blockchain technology, transaction immutability is often considered a fundamental principle. However, this rigidity presents challenges in real-world applications, particularly in financial transactions where errors, fraud, or unintended transfers can occur. Traditional banking systems allow reversals under specific conditions, ensuring a level of consumer protection that is absent in most cryptocurrency transactions. Therefore, implementing a mechanism for reversible transactions in blockchain-based systems is crucial to bridging this gap and fostering wider adoption.

Simply allowing all transactions to be reversible at any moment would be counterproductive, as it would undermine the core purpose of blockchain technology: ensuring finality and preventing double-spending. If transactions could be reverted indefinitely, users would lack trust in the system, and economic activity on the platform would be severely hampered.

To balance the need for security with the benefits of reversibility, time must be introduced as a condition for a transaction's refundability. However, a major technical challenge arises from the lack of precise timestamps in blockchain systems. Since miners can manipulate timestamps within a certain range, they are unreliable for enforcing time-dependent constraints. The one immutable and tamper-proof parameter available in a blockchain is the block number, making it the optimal choice for implementing a structured and predictable reversibility mechanism.

Methods and Concept

Transactions structure

The proposed solution relies on assigning a block limit to each transaction, determining the period during which it remains reversible. If a transaction's block limit expires, it becomes final and cannot be reverted. Each transaction must specify four key parameters: the **recipient** (destination address), the **amount** being transferred, the **block limit** (specifying how long the transaction remains refundable), and the **debt indices**, which are used for debt verification processes.

When a transaction occurs, four processes take place sequentially:

- **Minimal Refund Block Checking** – Ensures that the block limit adheres to the minimum allowed refundability period.
- **Debt Checking** – Verifies whether the sender has outstanding liabilities that affect transaction validity.
- **Balance Change** – Adjusts the sender's and recipient's token balances accordingly.
- **Storage Pushing** – Records the transaction details in the blockchain for future reference.

A fundamental aspect of this system is the concept of the **minimal refund block (MRB)**. This value is assigned to each address upon receiving a transaction for the first time, defaulting to 300 blocks. The minimal refund block guarantees a minimum refundability period unless explicitly changed by the user.

The rules governing the refund period are as follows:

- If the block limit is lower than the minimal refund block, it is adjusted to match the *minimal refund block*;
- If the *minimal refund block* is set to zero, transactions can be made immediately irreversible;
- By default, the *minimal refund block* is set to 300 blocks (arbitrary value).

A dedicated function, `changeMinimalRefundBlock()`, allows users to modify their minimal refund block value. This function operates in two stages: (1) initiating a countdown equal to the current minimal refund block and (2) finalizing the change once the countdown expires. The process can be canceled at any time using `cancelMinimalRefundBlockChange()` to prevent accidental modifications.

Minimal Refund Block Change Process

The Minimal Refund Block Change Process is a crucial mechanism that allows users to modify the period in which transactions remain reversible. This process is implemented through the `changeMinimalRefundBlock(value)` method, which governs the update of the Minimal Refund Block (MRB) associated with an address.

The `changeMinimalRefundBlock(value)` function accepts a single input parameter: the new **MRB** value to be assigned to the address. However, the process is not instantaneous. Instead, it follows a structured time-dependent procedure to prevent abrupt changes that could compromise the transaction reversal mechanism.

- Initial Call:** When `changeMinimalRefundBlock(value)` is called, a countdown is initiated, starting from the current block number at the moment of execution.
- Countdown Period:** The countdown lasts for a duration equal to the latest **MRB** of the address, ensuring a gradual transition.
- Completion:** Once the countdown reaches completion, the user can call `changeMinimalRefundBlock(value)` again, at which point the **MRB** is officially updated to the originally specified value.
- Enabling Irreversible Transactions:** If the **MRB** is set to zero, the address will be able to perform non-reversible transactions, meaning that funds sent from this address may not be eligible for a refund (if user choose so).

This delayed execution serves as a safeguard, preventing immediate adjustments that could be exploited for fraudulent activity. That's called a *Private key stealing protective measure*.

To provide additional control, the system includes a cancellation mechanism. At any moment during the waiting period, the address owner can invoke `cancelMinimalRefundBlockChange()`, which terminates the ongoing countdown. This feature is particularly useful in cases where a user mistakenly initiates a change or recognizes the need to retain the existing **MRB** setting.

Private key stealing intrisec protective measure

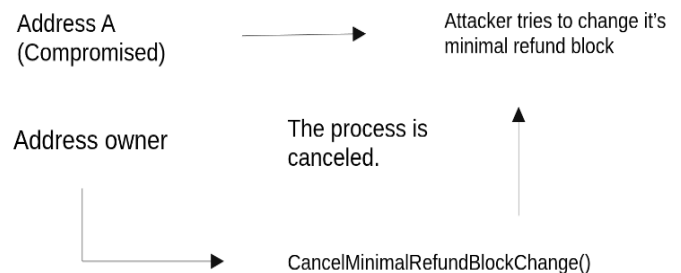


Figure 1. Private key stealing protective measure example.

The delayed execution of MRB changes serves several critical purposes:

- **Mitigation of Fraud Risks:** Without a waiting period, users could instantly switch between refundable and non-refundable states, creating opportunities for malicious activities such as withdrawing funds after a fraudulent transaction.
- **Stability and Predictability:** The structured delay ensures that transaction reversibility policies remain stable and predictable for all network participants.
- **Defense Against Miners' Manipulation:** Since the process relies on block numbers rather than timestamps, miners cannot artificially accelerate or delay the **MRB** change.

Debt Checking

A critical vulnerability in the refundable transaction model is the *"send before refund"* flaw. This occurs when a recipient of a refundable transaction transfers the received tokens to another address before the refund period expires. This action allows malicious users to circumvent the refund mechanism, effectively stealing tokens and undermining the integrity of the refundable transaction framework.

To address this issue, a robust mechanism called *debt checking* is implemented. *Debt checking* ensures that an address cannot send funds that are still subject to potential refunds, thereby preventing unauthorized fund transfers that would invalidate the refund mechanism. This process comes with minimal computational overhead while significantly strengthening transaction security.

Debt checking, combined with minimal refund block constraints, ensures a robust reversible transaction mechanism while maintaining the usability and efficiency of the system. The implementation of these safeguards upholds the token's integrity, preventing fund manipulation and unauthorized transfers, thereby making refundable transactions both secure and practical.

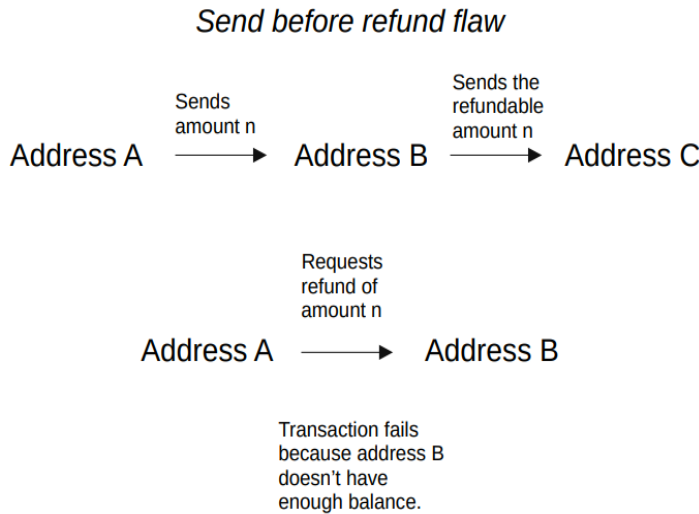


Figure 2. A send before refund attack.

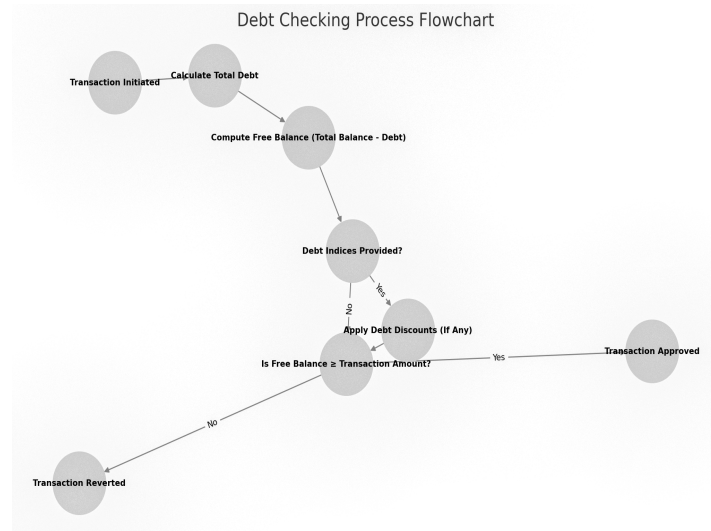


Figure 3. Debt checking algorithm.

Mechanism of Debt Checking

Debt checking fundamentally operates by verifying the sender's free balance, which is the total balance of an address minus any amounts that remain refundable to other addresses. This prevents users from transferring funds that are still subject to potential refund claims.

Every transaction must go through the following verification steps before it is processed:

- i) **Debt Calculation:** The system identifies all refundable transactions associated with the sender's address and calculates the total debt, which represents the sum of tokens that could still be reverted.
- ii) **Free Balance Calculation:** The free balance is computed by subtracting the sender's total debt from their current balance. This represents the maximum amount the sender can transfer without compromising the refundability of previous transactions.
- iii) **Debt Indices Variable:** A critical variable, the debt indices, must be assigned to the transferring method. This array, set by the sender, contains indices of refunds that are no longer valid. This allows for expired refundable amounts to be discounted, reducing the total recorded debt and ensuring that past refundable transactions do not indefinitely restrict an address's free balance. The reason this is user specified is to maintain constant operation time, and thus reducing GAS cost.
- iv) **Transaction Verification:** Before processing a transaction, the protocol verifies if the free balance is sufficient to cover the transaction amount. If the sender's free balance is less than the intended transfer amount and the debt indices are not specified, the transaction is automatically reverted, preventing a potential send before refund attack.
- v) **Debt Discounting:** If debt indices are specified, the system recalculates the sender's debt by discounting the amounts corresponding to expired refunds. If the discounted debt still results in a free balance lower than the transaction amount, the transaction is reverted.

Refunds System

In the **Refundable Token (RFT)** system, refunds are a crucial feature that allows for transactions to be reversible within a defined period, providing users with the flexibility to cancel or amend their transactions based on specific conditions. The effective storage and management of refunds are integral to the success of the system. This section delves into the underlying architecture for refund storage and how the system ensures proper tracking and execution.

General Refunds Structure

A refund in the RFT system is a data structure that holds information regarding a transaction that is eligible for reversal, along with its conditions for being refunded. This data structure is defined as follows, corresponding to a transaction:

```

struct Refund {
    address issuer;
    uint128 amount;
    uint128 blockEnd;
}
  
```

The Refund structure contains three key fields:

- i) **issuer:** This is the address of the sender of the original transaction. The issuer is responsible for initiating the refund.
- ii) **amount:** This represents the value of the transaction that can be refunded within the allowed time window.
- iii) **blockEnd:** The block number after which the refund is no longer valid. This field ensures that refunds are time-bound and can only be processed within a specified period, thus maintaining the integrity of the system.

Refund Storage Structure

Refunds are stored in a central mapping variable called `_addrTransactionsRefunds`, which is associated with the recipient of each transaction. Each recipient's refunds are stored as an array of Refund objects, making it possible to track multiple refunds for a single recipient. This structure ensures that each recipient can have several refundable transactions associated with them.

```
mapping(address => Refund[]) _addrTransactionsRefunds;
```

The array structure enables each recipient to store all the refunds linked to their address. *Refunds* can be cleared either partially or fully, depending on the amount refunded. If a recipient receives a partial refund, the remaining amount in the *Refund* object is updated, and the recipient's total debt is adjusted accordingly. Transactions that contain the block limit variable set as zero will not push a refund object to the recipient array.

The refund storage system within the Refundable Token framework plays a vital role in ensuring that the transactions remain flexible, time-bound, and secure. By storing refunds as structured objects in a recipient's transaction array, the system facilitates efficient tracking of refundable amounts and associated conditions.

The Refund Method

The refund process is handled through the `getRefund(recipient, id)` method. This method allows the issuer of a transaction to recover the amount from a previously made refundable transaction. It is designed to ensure that the transaction is refundable within a specific block range and that only the issuer of the transaction can request the refund.

The `getRefund` method requires two parameters:

- recipient:** the address of the individual who received the initial transaction.
- id:** the unique identifier of the transaction within the recipient's refund storage.

In essence, this method allows the issuer of a transaction to request a refund of a previously transferred amount, given that the transaction is still within the refundable period. Once a refund is requested, the `getRefund(recipient, id)` method verifies that the transaction is still refundable by checking the following conditions:

- The sender of the refund request must be the issuer of the transaction, ensuring that only the original sender can initiate the refund.
- The refund request must be within the valid timeframe, meaning that the current block number must be less than the `blockEnd` value.
- The requested refund amount cannot exceed the original amount stored in the transaction object.

If these conditions are met, the refund amount is transferred back to the issuer's address, and the transaction is removed from the recipient's refund array, effectively marking the transaction as completed. If the refund is partial, the amount is reduced accordingly, and the transaction object is updated with the remaining balance.

The implementation of the `getRefund` method is crucial in ensuring the integrity and security of the system. By enforcing these checks, the method

guarantees that refunds are only processed when appropriate and prevents abuse of the refund mechanism. Furthermore, by storing refunds directly on the blockchain, the system ensures transparency and traceability of all refundable transactions, providing a robust mechanism for handling transactions in a decentralized environment.

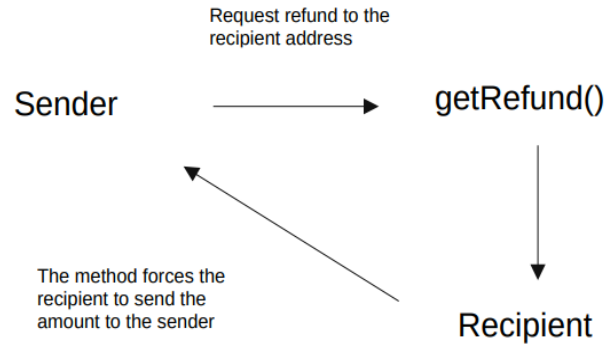


Figure 4. The refund method process function calling.

Results

The implementation of the RFT was thoroughly tested in a *Truffle-Ganache environment* to validate its functionality and performance. All tests were successfully passed, confirming that the contract handles both refundable and non-refundable transactions as intended. In particular, key operations such as creating a refundable transaction, refunding an existing transaction, changing the minimal refund block, and managing debt checks were verified by using the *Mocha Javascript Testing* framework.

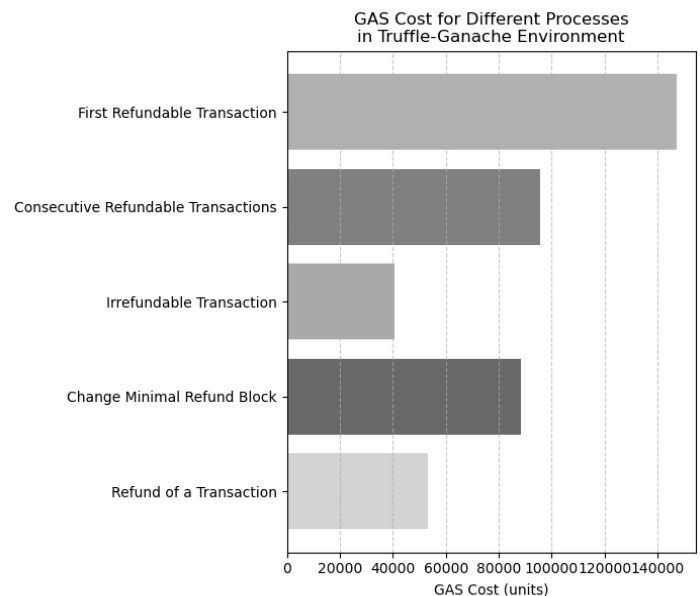


Figure 5. GAS cost per address of each process.

As Figure 5 illustrates the gas costs of each primary operation measured in the *Ganache* test suite. These operations exhibit constant-time $\mathcal{O}(1)$ complexity in their respective methods, meaning they do not scale with the size of the array or the number of users. This efficiency is attributed to the direct mapping approach in Solidity, where refunds are indexed and accessed by fixed-size arrays and keys, ensuring minimal computational overhead.

Overall, the successful test results and stable gas costs indicate that the RFT contract can be integrated reliably into decentralized applications requiring refundable transactions. The constant-time operations ensure predictable performance, and the mechanism of block-based refundability adds a flexible yet secure layer to typical token transfers. This confirms the feasibility, reliability, and cost-effectiveness of the approach, paving the way for broader adoption of time-ensured electronic refund systems in blockchain environments.